

Documento de Arquitetura

1. Arquitetura Escolhida

A solução adota uma arquitetura híbrida containerizada baseada em **microsserviços** e **invocação de processos nativos**.

A camada de exposição dos serviços foi desenvolvida utilizando **ASP.NET Core 8 (.NET 8 / C#)**, disponibilizando uma **API RESTful** responsável por:

- Receber requisições HTTP em formato JSON;
- Validar os dados de entrada;
- Traduzir as informações para o formato esperado pelo sistema legado;
- Invocar o programa COBOL localmente por meio de chamadas ao sistema operacional (**Standard Input/Output**).

O processamento das regras de negócio e a persistência dos dados permanecem exclusivamente no ambiente **COBOL**, utilizando arquivos indexados como mecanismo de armazenamento.

2. Justificativas Técnicas

2.1 Isolamento de Responsabilidades

A utilização de uma API .NET como camada intermediária permite que aplicações web, mobile ou integrações com terceiros consumam os serviços utilizando padrões modernos (REST/JSON), sem necessidade de conhecer detalhes da implementação em COBOL.

2.2 Preservação do Sistema Legado

A API utiliza a classe **Process.Start** para executar o binário COBOL, preservando integralmente a lógica de negócio existente e reduzindo riscos de inconsistência ou corrupção dos dados.

2.3 Contrato de Comunicação

A comunicação entre o C# e o COBOL ocorre por meio de uma **string posicional de tamanho fixo (99 bytes)**.

Esse formato reproduz o comportamento tradicional de um **Copybook COBOL**, garantindo:

- Estrutura fixa de dados;
- Compatibilidade entre as linguagens;
- Previsibilidade na troca de informações;
- Baixo acoplamento entre as camadas.

2.4 Portabilidade

Toda a aplicação é executada em um único contêiner **Docker**, utilizando uma estratégia de **multi-stage build**.

Essa abordagem proporciona:

- Padronização do ambiente de execução;
 - Facilidade de implantação;
 - Compatibilidade com pipelines de CI/CD;
 - Possibilidade de implantação em ambientes locais ou em nuvem.
-

3. Fluxo de Execução da Solução

1. Interação do usuário

2. O atendente acessa a interface web e informa os dados do cliente.

3. Requisição HTTP

4. O navegador envia uma requisição JSON para a API REST.

5. Validação

6. A API realiza validações básicas (CPF, tamanho dos campos e obrigatoriedade dos dados).

7. Conversão

8. O Service converte o JSON para uma string posicional de 99 caracteres.

9. Execução do COBOL

10. O processo .NET executa o programa GnuCOBOL, enviando a string como entrada.

11. Processamento

12. O programa COBOL executa a operação solicitada (**Consultar, Inserir, Atualizar ou Excluir**) diretamente sobre o arquivo indexado (`clientes.dat`).

13. Retorno

14. O COBOL devolve um código de status juntamente com os dados processados.

15. Resposta da API

16. A API interpreta o retorno do COBOL e converte o resultado em um código HTTP apropriado, como:

- **200 OK**
- **201 Created**
- **400 Bad Request**
- **404 Not Found**
- **500 Internal Server Error**

17. Atualização da Interface

18. A interface recebe a resposta da API e atualiza a tela conforme o resultado da operação.

4. Componentes Utilizados

Camada	Tecnologia
Backend / API	ASP.NET Core 8 (.NET 8 – C#)
Arquitetura	RESTful API com Injeção de Dependência (IoC)
Sistema Legado	GnuCOBOL
Persistência	Arquivos Indexados (simulação de VSAM)
Frontend	HTML5, JavaScript (Fetch API) e Bootstrap 5
Testes	xUnit e Moq
Infraestrutura	Docker (multi-stage build) utilizando imagens Linux (Debian/Alpine)